SAN JOSÉ STATE UNIVERSITY · CMPE 172 ENTERPRISE SOFTWARE PLATFORMS

TutorBook

Term Project, Milestone 3 Report
Integration, Distribution & Deployment

Student	Samuel Dinkayehu

Scenario	Tutoring center appointment booking
Stack	Java 17+ · Spring Boot 4 · SQL over JDBC (no ORM) · H2 · Micrometer · Docker · GitHub Actions
GitHub	https://github.com/samuel1gg/cmpe172-tutorbook
Date	October 27, 2026

Contents

1. What Milestone 3 delivers
2. Distribution boundary: diagram, coarse-grained interface and DTO, mock notification service
3. Fault handling: timeout, retry, fallback, idempotency, commit-then-notify
4. System management: logging plan, health endpoint, metrics
5. Failure scenario and recovery strategy
6. Delivery: Dockerfile, docker-compose, GitHub Actions CI
7. Testing
8. How to run

1. What Milestone 3 Delivers

Requirement	Where it is implemented
Mock notification service (second Spring controller; accepts a DTO with recipient, type, message; stores it; returns success)	`MockNotificationController` + `MockNotificationService` at `/mock-notification-service`
Main app calls it through a Gateway with a coarse-grained DTO	`NotificationGateway` (interface) → `HttpNotificationGateway`; DTO `NotificationRequest`
Distribution diagram: Client → Appointment Scheduler → Notification Service	Section 2.1, `docs/Distribution_Diagram.png`
Timeout, retry and fallback; booking succeeds if the service is slow or down	`HttpNotificationGateway`, `NotificationDispatcher` (Section 3)
Remote call outside the DB transaction (commit, then notify); idempotent retries	`AppointmentEvent` + `AppointmentNotifier` (`@TransactionalEventListener(AFTER_COMMIT)`); `Idempotency-Key`, `UNIQUE(idempotency_key)`
Confirmation on both book and cancel	`BookingTransaction.tryBook` → `BOOKING_CONFIRMED`; `AppointmentService.cancel` → `BOOKING_CANCELLED`
Logging plan with INFO / WARN / ERROR examples	Section 4.1
`GET /health` with a database-connectivity check	`HealthController` (Section 4.2)
At least one metric	5 metrics: bookings by outcome, booking latency, bookings in the last hour, notifications by outcome, notification backlog (Section 4.3)
One failure scenario and its recovery strategy	Notification service outage → `FAILED` → `NotificationRetryJob` (Section 5)
Dockerfile + docker-compose.yml	Multi-stage `Dockerfile` with `HEALTHCHECK`; `docker-compose.yml` runs the app (H2 is embedded, like SQLite)

GitHub Actions: build and test on every push	`.github/workflows/ci.yml` (test job + Docker build and /health smoke test)
Integration test: booking succeeds when the notification stub is down	`NotificationResilienceTest.bookingSucceeds_whenNotificationServiceIsDown` and `NotificationServiceUnreachableTest`

This notification service is the same channel Milestone 4 will reuse for AI-generated reminders. The `NotificationType` enum already has a `REMINDER` value, and the agent will only need to build a `NotificationRequest`.

2. Distribution Boundary

2.1 Distribution diagram

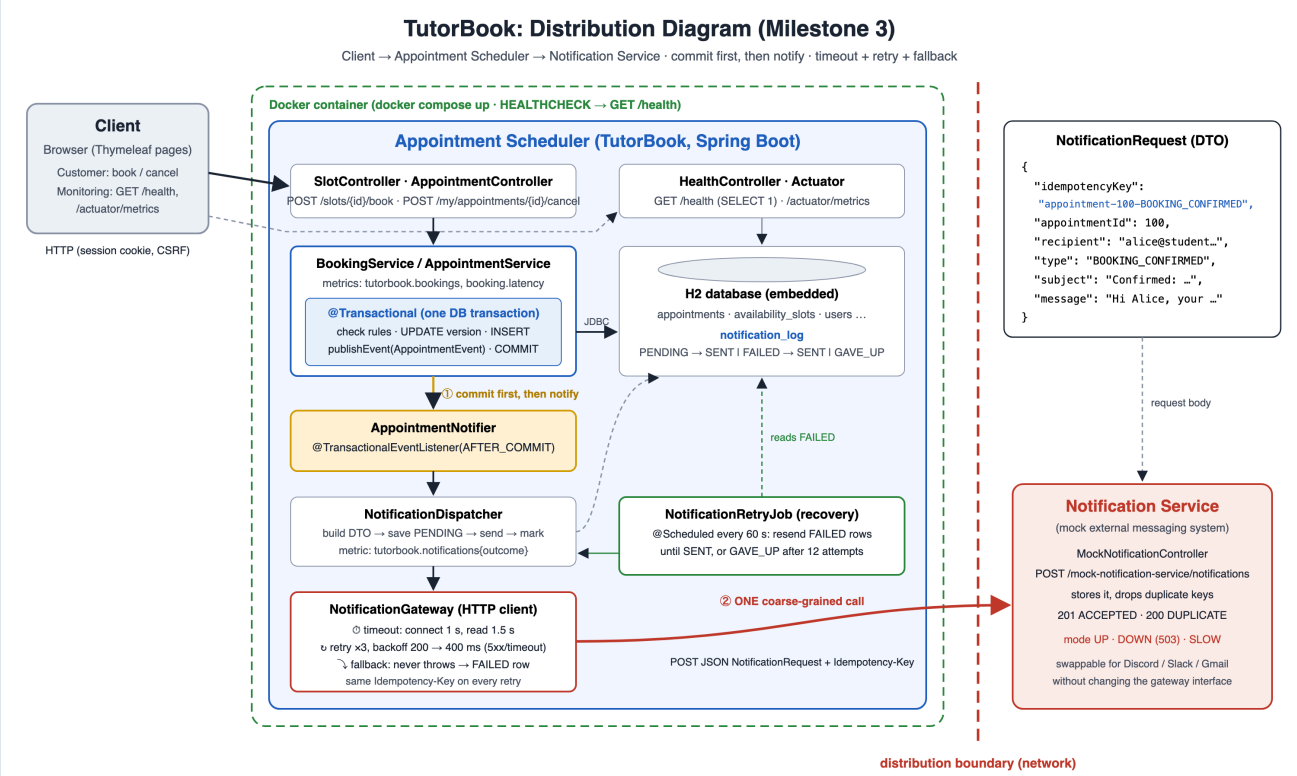


Figure 1. Client → Appointment Scheduler → Notification Service. The red dashed line is the distribution boundary; only one coarse-grained HTTP call crosses it.

1. **Client → Appointment Scheduler.** The browser posts the booking form (session cookie + CSRF token) to `SlotController`, exactly as in Milestone 2.
2. **Inside the scheduler.** `BookingTransaction.tryBook` runs one `@Transactional` unit: check rules, bump the slot version, insert the appointment, and *publish* an `AppointmentEvent`. Nothing is sent yet.
3. **After COMMIT.** Spring delivers the event to `AppointmentNotifier`. `NotificationDispatcher` builds the DTO, saves it as `PENDING` in `notification_log`, and hands it to the gateway.
4. **Appointment Scheduler → Notification Service.** `HttpNotificationGateway` makes **one** HTTP `POST` with the JSON DTO and an `Idempotency-Key` header, protected by a timeout, retries and a fallback.

2.2 The coarse-grained interface and DTO

```
/** The DTO that crosses the distribution boundary (TutorBook -> Notification Service). */
public record NotificationRequest(
        @NotBlank @Size(max = 100) String idempotencyKey,   // "appointment-12-BOOKING_CONFIRMED"
        @NotNull Long appointmentId,
        @NotBlank @Size(max = 255) String recipient,        // the customer's email
        @NotNull NotificationType type,                     // CONFIRMED / CANCELLED / REMINDER
        @NotBlank @Size(max = 200) String subject,
        @NotBlank @Size(max = 1000) String message) { }     // the finished, human-readable text

public interface NotificationGateway {
    SendResult send(NotificationRequest request);           // never throws
}
```

Why coarse-grained. A fine-grained design would let the notification service call back into TutorBook: `getCustomer(id)`, `getSlot(id)`, `getProvider(id)`, then `send()`. That is four network round trips per notification. Each one adds latency, each one can fail on its own (partial failure), and the remote service would become coupled to TutorBook's internal model. Instead, TutorBook does all the lookups locally (one SQL join in `AppointmentRepository.findDetailsById`) and sends **one self-contained DTO**:

- **One round trip**, so there is one place to apply a timeout and one thing to retry.
- **No shared state.** The service needs nothing from our database, so it can live on another machine or be a third-party product.
- **Stable contract.** The DTO is a small public record; our tables can change without breaking the remote service.
- **Replaceable.** Pointing `notification.base-url` at a Discord/Slack webhook adapter or a Gmail SMTP service changes nothing else, because the rest of the app only knows `NotificationGateway`.

The DTO is built in one place, `NotificationDispatcher.build(...)`, for example: *"Hi Alice Nguyen, your Calculus Tutoring session with Dr. Priya Patel on Wed, Oct 28 at 9:00 AM is confirmed. Appointment #100."*

2.3 The mock notification service

`MockNotificationController` is a second `@RestController` that stands in for an external messaging system. TutorBook never calls it as a Java method; it is reached only over HTTP through the gateway, exactly like a real remote service.

```
@PostMapping("/notifications")
public ResponseEntity<NotificationResponse> receive(
        @RequestHeader(value = "Idempotency-Key", required = false) String idempotencyKey,
        @Valid @RequestBody NotificationRequest request) throws InterruptedException {
    service.countRequest();
    switch (service.mode()) {
        // simulate an outage / a slow service
        case DOWN -> { return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).build(); }
        case SLOW -> Thread.sleep(service.slowDelayMs());
        case UP -> { }
    }
    Delivered d = service.accept(request);          // stores it (null = duplicate key)
    if (d == null) {
        Delivered first = service.find(request.idempotencyKey());
        return ResponseEntity.ok(
                new NotificationResponse(first.notificationId(), "DUPLICATE", first.receivedAt()));
    }
    log.info("[mock-notification-service] {} {} -> {}: {}", d.notificationId(), request.type(),
            request.recipient(), request.subject());
    return ResponseEntity.status(HttpStatus.CREATED)
            .body(new NotificationResponse(d.notificationId(), "ACCEPTED", d.receivedAt()));
}
```

Endpoint	Purpose
`POST /mock-notification-service/notifications`	Receive one notification. `201 ACCEPTED` (new), `200 DUPLICATE` (key already seen), `400` (invalid DTO), `503` (mode DOWN)
`GET /mock-notification-service/notifications`	List what it has "sent" (used in the demo)
`PUT /mock-notification-service/mode/{UP|DOWN|SLOW}`	Simulate an outage or a slow service at runtime

3. Fault Handling

3.1 Timeout, retry and fallback in the gateway

```
public SendResult send(NotificationRequest request) {
    long backoffMs = props.initialBackoff().toMillis();                 // 200 ms
    String lastError = null;
    for (int attempt = 1; attempt <= props.maxAttempts(); attempt++) {   // 3 attempts
        try {
            // TIMEOUT: connect 1 s, read 1.5 s (set on the HTTP client)
            NotificationResponse response = restClient.post()
                    .uri(baseUrl() + PATH)
                    .contentType(MediaType.APPLICATION_JSON)
                    .header("Idempotency-Key", request.idempotencyKey())  // same key on every attempt
                    .body(request)
                    .retrieve()
                    .body(NotificationResponse.class);
            return SendResult.delivered(attempt);
        } catch (HttpClientErrorException e) {
            if (e.getStatusCode().value() != 429) {                        // 4xx: our request is wrong,
                return SendResult.failed(attempt, false, "HTTP " + ...);   // retrying cannot fix it
            }
            lastError = "HTTP 429";
        } catch (RestClientResponseException e) {
            lastError = "HTTP " + e.getStatusCode().value();               // 5xx: retry
        } catch (RestClientException e) {
            lastError = describe(e);                           // timeout / refused: retry
        }
        if (attempt < props.maxAttempts()) {
            log.warn("Notification {} attempt {}/{} failed ({}); retrying in {} ms", ...);
            sleep(backoffMs);
            backoffMs *= 2;                                    // RETRY, exponential backoff
        }
    }
    log.error("Notification {} failed after {} attempts ({}); booking is kept, will retry later", ...);
    return SendResult.failed(props.maxAttempts(), true, lastError);       // FALLBACK: never throws
}
```

What goes wrong	Detected as	Retried?	Result for the user
Service down / not deployed	connection refused	yes, 3×	Booking succeeds; notification `FAILED`, retried by the job
Service overloaded or crashing	HTTP 503 / 500 / 429	yes, 3×	Same
Service slow (hangs)	read timeout after 1.5 s	yes, 3×	Booking succeeds within about 5 s worst case (3 × 1.5 s + 0.6 s backoff)
Our DTO is invalid	HTTP 400	no	Booking succeeds; notification `GAVE_UP` and logged as ERROR

The settings are in `application.properties` (`notification.connect-timeout`, `read-timeout`, `max-attempts`, `initial-backoff`) and bound to the `NotificationProperties` record. The timeouts are short because a confirmation is not worth making the customer wait: the booking is already committed, and a late notification can still be delivered by the retry job.

3.2 Fallback: the dispatcher never lets a notification break a booking

```
/** Called after a booking or cancel has committed. Never throws. */
public void notify(long appointmentId, NotificationType type) {
    try {
        // short transaction: build the DTO and save it as PENDING
        Optional<NotificationRequest> request =
                newTransaction.execute(s -> prepare(appointmentId, type));
        if (request != null && request.isPresent()) {
            deliver(request.get(), 0);       // network call: OUTSIDE any transaction
        }
    } catch (RuntimeException e) {
        log.error("Could not create {} notification for appointment {}", type, appointmentId, e);
        count(type, "error");
    }
}

private void deliver(NotificationRequest request, int attemptsSoFar) {
    SendResult result = gateway.send(request);
    if (result.delivered()) { markSent(...); count(type, "sent"); return; }
    boolean giveUp = !result.retryableLater() || total >= props.maxTotalAttempts();
    // fallback: keep a durable record instead of losing the notification
    markFailed(key, result.attempts(), giveUp ? "GAVE_UP" : "FAILED", result.error());
}
```

The fallback is **graceful degradation with a durable record**. The customer still gets the confirmation page (the booking is real), and the missed notification is saved in `notification_log` instead of being lost, so it can be delivered later (Section 5).

3.3 Commit first, then notify

The remote call must not be inside the database transaction. If it were:

- **Ghost notifications.** The service could send "your booking is confirmed" and then the transaction could still roll back (for example, the unique-constraint backstop fires). The customer would be told about a booking that does not exist.
- **Long transactions.** A slow service (up to about 5 s) would keep the transaction and its row locks open the whole time, blocking other bookings of that slot.
- **Coupled failure.** An exception from the HTTP client could roll back a perfectly good booking.

So the transaction only *publishes an event*, and Spring delivers it after the commit:

```
// BookingTransaction.tryBook(...)  -- inside @Transactional(isolation = READ_COMMITTED)
appointmentId = appointmentRepository.insertBooked(slotId, customerId, notes);
// 5. ask for a confirmation; delivered only AFTER this transaction commits
events.publishEvent(new AppointmentEvent(appointmentId, NotificationType.BOOKING_CONFIRMED));
return appointmentId;

// AppointmentService.cancel(...)  -- inside @Transactional
if (appointmentRepository.cancel(appointmentId) == 0) { throw new SlotConflictException(...); }
events.publishEvent(new AppointmentEvent(appointmentId, NotificationType.BOOKING_CANCELLED));

@Component
public class AppointmentNotifier {
    // runs only if the commit succeeded
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onAppointmentChanged(AppointmentEvent event) {
        dispatcher.notify(event.appointmentId(), event.type());
    }
}
```

If the transaction rolls back (for example, a version conflict that leads to a retry, or a 409), the event is discarded and no notification is sent. With optimistic retries, only the attempt that commits produces exactly one notification. Because the listener runs inside the `afterCommit` callback of the finished transaction, the dispatcher's own database writes use `PROPAGATION_REQUIRES_NEW` (two tiny transactions: save `PENDING`, then mark the result), and the HTTP call sits between them with no transaction open.

3.4 Idempotent retries

Retries are only safe if repeating a request cannot create a second notification. A timeout is ambiguous: the request may have arrived and been processed, and only the answer was slow. TutorBook therefore gives every notification a deterministic key and sends it on every attempt:

Layer	Mechanism
Key	`"appointment-" + id + "-" + type`, for example `appointment-100-BOOKING_CONFIRMED`. Book and cancel have different keys, so a cancellation is never mistaken for a duplicate.
TutorBook database	`notification_log.idempotency_key` is `UNIQUE`, so the same notification cannot be created twice (e.g. a duplicated event).
Remote service	The mock stores each key once; a repeat gets `200 DUPLICATE`, which the gateway treats as delivered.
Retry job	Claims a row with a compare-and-set (`UPDATE … SET status='PENDING' WHERE key=? AND status='FAILED'`) so two runs never send the same row at the same time.

The test `bookingSucceeds_whenNotificationServiceIsSlow_andRetriesDoNotDuplicate` proves this: the slow service receives all 3 timed-out attempts but keeps **exactly one** notification, and the later retry is answered `DUPLICATE` and marked `SENT`.

4. System Management and Observability

4.1 Logging plan

What we log: every state change of a booking or notification (one line each, with the ids needed to trace it), every retry and fallback, health-check failures, and unexpected exceptions (stack trace in the server log only, never sent to the client). **What we do not log:** passwords, session ids, or full email addresses. Recipients are masked as `a***@student.sjsu.edu`. Logs go to stdout in one line per event (`time level [thread] class - message`), which is what `docker logs` and log collectors expect.

Level	When	Examples the code emits

INFO	Normal business events	`BookingService - Booked appointment 100 (slot 1, customer 1)`
		`NotificationDispatcher - BOOKING_CONFIRMED for appointment 100 sent to a***@student.sjsu.edu`
		`AppointmentService - Cancelled appointment 100 (customer 1)`
		`NotificationRetryJob - Retrying 1 failed notification(s)`

WARN	Something went wrong but the system handled it	`HttpNotificationGateway - Notification appointment-101-BOOKING_CONFIRMED attempt 1/3 failed (HTTP 503); retrying in 200 ms`
		`NotificationDispatcher - Notification appointment-101-BOOKING_CONFIRMED saved as FAILED after 3 attempts (HTTP 503); NotificationRetryJob will retry it`
		`BookingService - Slot 7 still contended after 3 attempts; returning 409`

ERROR	A request or a notification could not be completed; needs attention	`HttpNotificationGateway - Notification appointment-101-BOOKING_CONFIRMED failed after 3 attempts (HTTP 503); booking is kept, will retry later`
		`NotificationDispatcher - Gave up on notification appointment-7-BOOKING_CONFIRMED after 12 attempts (connection refused)`
		`HealthController - Health check: database is unreachable (CannotGetJdbcConnectionException)`

4.2 Health endpoint: `GET /health`

```
@GetMapping("/health")
public ResponseEntity<Map<String, Object>> health() {
    Map<String, Object> database = checkDatabase();            // runs SELECT 1, measures response time
    boolean dbUp = "UP".equals(database.get("status"));
    long waiting = dbUp ? notificationRepository.countByStatus("FAILED") : 0;
    String overall = !dbUp ? "DOWN" : waiting > 0 ? "DEGRADED" : "UP";
    ...
    return ResponseEntity.status(dbUp ? HttpStatus.OK : HttpStatus.SERVICE_UNAVAILABLE).body(body);
}
```

Status	HTTP	Meaning
`UP`	200	Database answers `SELECT 1`; no notifications waiting.
`DEGRADED`	200	Database fine (bookings work), but some notifications are waiting for a retry. Still 200, so a load balancer keeps sending traffic.

`DOWN`	503	Database unreachable; nothing works. Docker's `HEALTHCHECK` marks the container unhealthy.

```
{"status":"UP","timestamp":"2026-10-28T04:52:12.62Z","uptimeSeconds":2,
 "database":{"status":"UP","responseTimeMs":0},
 "notifications":{"status":"UP","waitingForRetry":0,"gaveUp":0}}

// while the notification service is DOWN:
{"status":"DEGRADED","timestamp":"2026-10-28T04:52:13.81Z","uptimeSeconds":3,
 "database":{"status":"UP","responseTimeMs":0},
 "notifications":{"status":"DEGRADED","waitingForRetry":1,"gaveUp":0}}
```

Spring's `/actuator/health` is still available as well; `/health` is our own endpoint because it reports the notification backlog, which actuator does not know about.

4.3 Metrics

Metric (Micrometer)	Type	Recorded in	Use
`tutorbook.bookings{outcome=success|conflict|not_found}`	counter	`BookingService.book`	Successful vs failed bookings; conflict rate shows contention
`tutorbook.booking.latency`	timer	`BookingService.book`	Average/max booking time (count, total time, max)
`tutorbook.bookings.last_hour`	gauge	`MetricsConfig` (SQL count)	Bookings per hour
`tutorbook.notifications{type, outcome=sent|failed|gave_up}`	counter	`NotificationDispatcher`	Notification success rate
`tutorbook.notifications.backlog`	gauge	`MetricsConfig`	Notifications waiting for a retry: alert if it keeps growing

```
Timer.Sample sample = Timer.start(meters);
String outcome = "error";
try {
    long appointmentId = bookWithRetry(customerId, slotId, notes);
    outcome = "success";
    return appointmentId;
} catch (SlotConflictException e) { outcome = "conflict"; throw e; }
  catch (NotFoundException e)     { outcome = "not_found"; throw e; }
finally {
    sample.stop(latency);
    meters.counter("tutorbook.bookings", "outcome", outcome).increment();
}
```

Read from the running container:

```
GET /actuator/metrics/tutorbook.bookings
{"name":"tutorbook.bookings","measurements":[{"statistic":"COUNT","value":2.0}],
 "availableTags":[{"tag":"outcome","values":["success"]}]}

GET /actuator/metrics/tutorbook.booking.latency
{"name":"tutorbook.booking.latency","baseUnit":"seconds","measurements":[
  {"statistic":"COUNT","value":2.0},{"statistic":"TOTAL_TIME","value":0.7195},
{"statistic":"MAX","value":0.6419}]}

GET /actuator/metrics/tutorbook.notifications
{"name":"tutorbook.notifications","measurements":[{"statistic":"COUNT","value":3.0}],
 "availableTags":[{"tag":"type","values":["BOOKING_CONFIRMED"]},{"tag":"outcome","values":
["sent","failed"]}]}
```

5. Failure Scenario and Recovery Strategy

> **Scenario: the notification service goes down while customers are booking.** Detection, containment and recovery happen automatically; no booking is lost and no confirmation is lost.

Step	What happens	Mechanism
1. Failure	The service starts answering 503 (or stops answering at all).	Simulated with `PUT /mock-notification-service/mode/DOWN`
2. Containment	A customer books: the booking commits first, then the gateway tries 3 times (200 ms, 400 ms backoff) and gives up. The customer sees the normal confirmation page.	Commit-then-notify, timeout, retry, fallback
3. Durable record	The notification is saved as `FAILED` with `attempts = 3` and `last_error = 'HTTP 503'`.	`notification_log`
4. Detection	`GET /health` turns `DEGRADED`, the `tutorbook.notifications.backlog` gauge rises, and WARN/ERROR lines appear in the log.	Health, metrics, logging
5. Recovery	When the service is back, `NotificationRetryJob` (every 60 s) claims the `FAILED` row and resends it with the same idempotency key → `SENT`. Health returns to `UP`.	`@Scheduled` retry job, idempotency
6. Long outage	After 12 total attempts the row becomes `GAVE_UP` and is logged at ERROR, so an operator can investigate instead of retrying forever.	`notification.max-total-attempts`

The complete scenario, recorded from the app running in Docker:

```
21:52:13.113 WARN  MockNotificationController – [mock-notification-service] mode set to DOWN
21:52:13.152 WARN  MockNotificationController – [mock-notification-service] DOWN: rejecting appointment-
101-BOOKING_CONFIRMED
21:52:13.154 WARN  HttpNotificationGateway – Notification appointment-101-BOOKING_CONFIRMED attempt 1/3
failed (HTTP 503); retrying in 200 ms
21:52:13.364 WARN  HttpNotificationGateway – Notification appointment-101-BOOKING_CONFIRMED attempt 2/3
failed (HTTP 503); retrying in 400 ms
21:52:13.785 ERROR HttpNotificationGateway – Notification appointment-101-BOOKING_CONFIRMED failed after
3 attempts (HTTP 503); booking is kept, will retry later
21:52:13.788 WARN  NotificationDispatcher – Notification appointment-101-BOOKING_CONFIRMED saved as
FAILED after 3 attempts (HTTP 503); NotificationRetryJob will retry it
21:52:13.789 INFO  BookingService – Booked appointment 101 (slot 3, customer 1)      <- user got 302 to
the confirmation page in 0.65 s
        ... GET /health -> {"status":"DEGRADED", ... "waitingForRetry":1}
        ... PUT /mock-notification-service/mode/UP
21:53:12.003 INFO  NotificationRetryJob – Retrying 1 failed notification(s)
21:53:12.006 INFO  NotificationDispatcher – Retrying notification appointment-101-BOOKING_CONFIRMED
(attempts so far: 3)
21:53:12.021 INFO  HttpNotificationGateway – Notification appointment-101-BOOKING_CONFIRMED delivered on
attempt 1 (service status ACCEPTED)
        ... GET /health -> {"status":"UP", ... "waitingForRetry":0}
```

Other failure: the database is down. This cannot be worked around, so the design makes it visible. `/health` returns `503 DOWN`, Docker's `HEALTHCHECK` marks the container unhealthy after 3 failed checks,

and `restart: unless-stopped` in compose restarts it. Booking requests fail fast with a friendly 500 page (no stack trace).

6. Delivery: Docker and CI

6.1 Dockerfile (multi-stage)

```
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN chmod +x mvnw && ./mvnw -B -q dependency:go-offline     # dependencies cached in their own layer
COPY src/ src/
RUN ./mvnw -B -q package -DskipTests                        # tests run in CI, not in the image build

FROM eclipse-temurin:17-jre                                 # small runtime image: JRE + one jar
WORKDIR /app
RUN useradd --system --uid 1001 tutorbook
COPY --from=build /app/target/tutorbook-*.jar app.jar
USER tutorbook                                              # not running as root
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
  CMD curl -fsS http://localhost:8080/health > /dev/null || exit 1
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app/app.jar"]
```

6.2 docker-compose.yml

TutorBook uses **H2 embedded** in the application (no separate database server, the same situation as SQLite), so compose runs only the app container. Configuration is passed as environment variables, so the same image can point at a different notification service.

```
services:
  tutorbook:
    build: .
    image: tutorbook:latest
    ports:
      - "8080:8080"
    environment:
      NOTIFICATION_BASEURL: ""           # notification.base-url; blank = the in-app mock service
      NOTIFICATION_MOCK_MODE: "UP"       # UP, DOWN or SLOW (simulate an outage)
      JAVA_TOOL_OPTIONS: "-Duser.timezone=America/Los_Angeles"
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:8080/health"]
      interval: 30s
      retries: 3
    restart: unless-stopped
```

Verified locally: `docker compose up --build` builds the image, the container reports `healthy`, logging in and booking over HTTP works, and the outage scenario in Section 5 was recorded from this container.

6.3 GitHub Actions: `.github/workflows/ci.yml`

```
name: CI
on:
  push:
  pull_request:
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: "17", cache: maven }
      - run: ./mvnw -B verify                   # compiles and runs all 72 tests
      - uses: actions/upload-artifact@v4        # test reports, even if a test fails
        if: always()
        with: { name: surefire-reports, path: target/surefire-reports/ }
  docker:
    needs: build-and-test                       # only if the tests passed
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t tutorbook:ci .
      - run: |                                  # smoke test: start the image, wait for GET /health
          docker run -d --name tutorbook -p 8080:8080 tutorbook:ci
          for i in $(seq 1 30); do
            if curl -fsS http://localhost:8080/health; then exit 0; fi
            sleep 2
          done
          docker logs tutorbook; exit 1
```

7. Testing

72 tests, all passing (`./mvnw test` ; 11 are new in Milestone 3). The notification tests run the app on a random port so the gateway goes through real HTTP, with short timeouts (connect 300 ms, read 400 ms) to keep the suite fast.

Test	What it proves
bookingSucceeds_whenNotificationServiceIsDown	Stub answers 503: the booking is `BOOKED` in the database, the gateway tried exactly 3 times, the notification is `FAILED` with `HTTP 503` , and the `failed` metric went up.
bookingSucceeds_whenNotificationServiceIsUnreachable	Nothing is listening at the configured URL (connection refused): the booking still succeeds; the notification is `FAILED` .
bookingSucceeds_whenNotificationServiceIsSlow_andRetriesDoNotDuplicate	Stub is slower than the read timeout: the booking succeeds in under 5 s; the slow service received 3 requests but kept 1 notification; the retry job then marks it `SENT` .
failedNotificationIsDeliveredByRetryJob_whenServiceComesBack	Recovery: `FAILED` → service back `UP` → retry job → `SENT` with 4 total attempts and one copy at the service.
bookAndCancel_eachSendOneConfirmation_whenServiceIsUp	Book sends `BOOKING_CONFIRMED` , cancel sends `BOOKING_CANCELLED` , both `SENT` , to the right recipient.
mockServiceIsIdempotent	Same key twice → `201 ACCEPTED` then `200 DUPLICATE` , stored once.
healthEndpointReportsDatabase	`GET /health` → 200 with `database.status = UP` .
bookingMetricsAreRecorded	`tutorbook.bookings{outcome=success}` +1, latency timer and bookings-last-hour gauge recorded.
NotificationDispatcherTest (3)	The DTO contains everything the service needs; book and cancel have different keys; emails are masked in logs.
BookingTransactionTest, AppointmentServiceTest (updated)	A booking and a cancel each publish exactly one event; a forbidden cancel publishes none.

Milestone 2 tests (61)	Still pass: concurrency (2 and 8 threads), RBAC, validation, error pages, pagination, double-booking guard.

```
/** THE REQUIRED TEST: notification stub is down, booking still succeeds. */
@Test
void bookingSucceeds_whenNotificationServiceIsDown() {
    mockService.setMode(Mode.DOWN); // every request gets 503
    double failedBefore = counter("tutorbook.notifications", "outcome", "failed");

    long id = bookingService.book(ALICE.userId(), 3, null);

    // the booking is committed...
    assertThat(appointmentStatus(id)).isEqualTo("BOOKED");
    // ...the gateway tried 3 times, then fell back to saving the notification as FAILED
    NotificationRecord n = notification(id, NotificationType.BOOKING_CONFIRMED);
    assertThat(n.status()).isEqualTo("FAILED");
    assertThat(n.attempts()).isEqualTo(3);
    assertThat(n.lastError()).isEqualTo("HTTP 503");
    assertThat(mockService.requestsReceived()).isEqualTo(3);
    assertThat(mockService.all()).isEmpty();
    assertThat(counter("tutorbook.notifications", "outcome", "failed")).isEqualTo(failedBefore + 1);
}
```

8. How to Run

```
git clone https://github.com/samuel1gg/cmpe172-tutorbook.git
cd cmpe172-tutorbook

./mvnw spring-boot:run                 # http://localhost:8080   (JDK 17+)
./mvnw test                            # 72 tests

docker compose up --build              # same app in Docker; http://localhost:8080/health
curl -X PUT localhost:8080/mock-notification-service/mode/DOWN    # simulate the outage
curl localhost:8080/mock-notification-service/notifications       # what was "sent"
curl localhost:8080/actuator/metrics/tutorbook.bookings
```

Customer login: `alice@student.sjsu.edu` / `password123`. Provider login: `j.kim@tutorbook.edu` / `password123`.